

PROJECT REPORT

on

AUTONOMOUS TAXI REINFORCEMENT LEARNING AGENT

Navigation, Pickup and Drop-off using Tabular Q-Learning

Submitted in partial fulfilment of the requirements for the course
[Course Name & Code]

Submitted by

S.No	Reg ID	Name
1	23MIP10049	Kummari Rohan yadav
2	23MIP10050	Shaion Dutta
3	23MIP10020	Shakir Ali Qadri
4	23MIP10002	Pranjal Gupta

Int MTech / Data Science

Under the guidance of
Mrs. Vandhana Shakya

VIT BHOPAL

October, 2026

Github Repo Link : <https://github.com/Kumran24/Autonomous-Taxi-RL-Agent>

ABSTRACT

Reinforcement Learning (RL) enables an agent to learn optimal behaviour through trial-and-error interaction with an environment. This project implements a tabular Q-Learning agent that learns to operate an autonomous taxi in the Gymnasium Taxi grid-world: the agent must navigate a 5×5 city grid, pick up a passenger from one of four designated locations, and deliver them to a chosen destination in as few steps as possible.

The agent maintains a Q-table of 500 states by 6 actions, updated after every step using the Bellman (temporal-difference) update rule with a learning rate $\alpha = 0.1$ and discount factor $\gamma = 0.99$. An ϵ -greedy policy with linear ϵ -decay ($1.0 \rightarrow 0.01$) balances exploration and exploitation across 10,000 training episodes. After training, the greedy policy was evaluated and rendered in the terminal. Over 200 greedy evaluation episodes the agent achieved a 100% success rate with a mean return of 7.97 and a mean of 13.03 steps. Learning curves, policy heatmaps and a frame-by-frame storyboard of the trained agent are included.

Keywords: Reinforcement Learning, Q-Learning, Bellman Equation, Epsilon-Greedy, Gymnasium, Taxi Environment, Markov Decision Process.

TABLE OF CONTENTS

1. Introduction.....	4
1.1 Background.....	4
1.2 Problem Statement.....	4
1.3 Objectives	4
1.4 Scope.....	4
2. Theoretical Background.....	5
2.1 Markov Decision Process	5
2.2 Q-Learning and the Bellman Update	5
2.3 Epsilon-Greedy Exploration	5
3. Environment Description.....	6
3.1 State Space.....	6
3.2 Action Space	6
3.3 Reward Structure	6
4. System Design and Implementation	7
4.1 Tools and Technologies	7
4.2 Hyperparameters	7
4.3 Algorithm.....	7
4.4 Core Code	7
4.5 Evaluation and Visualisation	8
4.6 Visualisation Module.....	8
5. Results and Discussion	9
5.1 Training Behaviour	9
5.2 Learned Values and Policy	10
5.3 Evaluation over 200 Episodes.....	11
5.4 Behaviour of the Trained Agent (Animation Storyboard).....	11
5.5 Discussion.....	13
6. Observations and Limitations	15
7. Conclusion and Future Scope	16
7.1 Conclusion	16
7.2 Future Scope	16
References.....	17
Appendix A: Project Files.....	18

1. Introduction

1.1 Background

Many real-world problems, such as robotic navigation, ride-hailing dispatch and game playing, involve sequential decision-making where the consequence of an action may only become visible many steps later. Reinforcement Learning addresses such problems by letting an agent interact with an environment, receive scalar rewards, and gradually improve its policy. Q-Learning is one of the most fundamental model-free RL algorithms: it learns the value of taking each action in each state without needing a model of the environment's dynamics.

1.2 Problem Statement

To design and train an RL agent that, starting with no knowledge of the map, learns to drive a taxi to a passenger's location, pick the passenger up, drive to the requested destination and drop the passenger off in the minimum number of steps, while avoiding illegal pickup and drop-off actions.

1.3 Objectives

- To implement Tabular Q-Learning from scratch using NumPy, without deep neural networks.
- To demonstrate the explore-versus-exploit trade-off using an ϵ -greedy policy with decay.
- To apply the Bellman update to learn an optimal action-value function for a 500-state MDP.
- To evaluate the trained greedy policy and visualise its behaviour in the terminal.

1.4 Scope

The project is limited to a single-agent, fully observable, discrete environment. The learned policy is stored in a lookup table; continuous or very large state spaces are outside the scope and are discussed as future work.

2. Theoretical Background

2.1 Markov Decision Process

The Taxi task is modelled as a Markov Decision Process (MDP) defined by the tuple (S, A, P, R, γ) , where S is the set of states, A the set of actions, P the transition function, R the reward function and $\gamma \in [0,1)$ the discount factor. The goal is to find a policy π that maximises the expected discounted return $G = \sum \gamma^t r_t$.

2.2 Q-Learning and the Bellman Update

Q-Learning estimates the optimal action-value function $Q^*(s,a)$. After taking action a in state s , observing reward r and next state s' , the estimate is updated as:

$$Q(s,a) \leftarrow (1 - \alpha) \cdot Q(s,a) + \alpha \cdot [r + \gamma \cdot \max_{a'} Q(s',a')]$$

Here α is the learning rate, controlling how strongly new information overrides old, and γ is the discount factor, controlling how much future rewards matter relative to immediate ones. Because the target uses the maximum over next actions regardless of the action actually taken, Q-Learning is an off-policy algorithm.

2.3 Epsilon-Greedy Exploration

To learn, the agent must try actions it is not yet sure about (exploration) while also using what it already knows (exploitation). With probability ϵ the agent chooses a random action; otherwise it chooses $\operatorname{argmax}_a Q(s,a)$. Starting with $\epsilon = 1.0$ and decaying it over time moves the agent from pure exploration towards near-pure exploitation.

3. Environment Description

The project uses the Taxi environment from the Gymnasium library (the README refers to Taxi-v3; the notebook calls Taxi-v4 — see Section 6). The world is a 5×5 grid with internal walls and four designated stops: R (Red), G (Green), Y (Yellow) and B (Blue).

3.1 State Space

There are 500 discrete states, formed by the combination of 25 taxi positions, 5 passenger locations (four stops plus “inside the taxi”) and 4 destinations ($25 \times 5 \times 4 = 500$). Each state is encoded as a single integer used to index the Q-table.

3.2 Action Space

Action	Meaning	Description
0	South	Move one cell down (blocked by walls/edges)
1	North	Move one cell up
2	East	Move one cell right
3	West	Move one cell left
4	Pickup	Pick up the passenger if the taxi is at their location
5	Dropoff	Drop off the passenger if at the destination

Table 1: Discrete action space (6 actions)

3.3 Reward Structure

Event	Reward
Every ordinary step	-1 (encourages short routes)
Successful drop-off at the destination	+20 (episode terminates)
Illegal pickup or drop-off	-10

Table 2: Reward signal of the Taxi environment

4. System Design and Implementation

4.1 Tools and Technologies

Component	Detail
Language	Python 3 (notebook kernel: Python 3.13)
Libraries	gymnasium, numpy, matplotlib, random, time, os (pygame for graphical rendering)
Platform	Jupyter Notebook (taxiagent_with_visualizations.ipynb) / command-line script
Visualisation	Matplotlib plots; Gymnasium ANSI/RGB render modes for the animation

Table 3: Technology stack

4.2 Hyperparameters

Parameter	Value	Role
alpha (α)	0.1	Learning rate
gamma (γ)	0.99	Discount factor
epsilon (initial)	1.0	Begin with 100% random actions
epsilon_decay	0.001	Subtracted from ϵ after every episode
min_epsilon	0.01	Lower bound on exploration
episodes	10,000	Number of training episodes

Table 4: Training hyperparameters

4.3 Algorithm

- Create the Taxi environment and initialise a 500×6 Q-table with zeros.
- For each episode, reset the environment to obtain the initial state.
- Select an action: random with probability ϵ , otherwise argmax of $Q[\text{state}]$.
- Execute the action; observe the new state, reward and termination flag.
- Update $Q[\text{state}, \text{action}]$ using the Bellman rule of Section 2.2.
- Record the episode reward, step count and current ϵ for later plotting.
- Move to the new state; repeat until the episode ends.
- Decay ϵ linearly: $\epsilon \leftarrow \max(\epsilon_{\min}, \epsilon - 0.001)$.
- After training, run the greedy policy (no exploration) and render each step.

4.4 Core Code

```
for episode in range(episodes):
    state, info = env.reset()
    done = False; total_reward = 0; steps = 0
    while not done:
        if random.uniform(0, 1) < epsilon:
```

```

        action = env.action_space.sample()      # explore
    else:
        action = np.argmax(q_table[state, :])    # exploit

    new_state, reward, terminated, truncated, info = env.step(action)
    done = terminated or truncated

    old_value = q_table[state, action]
    next_max = np.max(q_table[new_state, :])
    q_table[state, action] = (1 - alpha) * old_value \
        + alpha * (reward + gamma * next_max)

    state = new_state
    total_reward += reward; steps += 1

episode_rewards.append(total_reward); episode_steps.append(steps)
epsilon_history.append(epsilon)
epsilon = max(min_epsilon, epsilon - epsilon_decay)

```

Listing 1: Training loop (from taxiagent.ipynb)

4.5 Evaluation and Visualisation

After training, the environment is re-created with `render_mode="human"` and then with `render_mode="ansi"`. At each step the agent selects the greedy action $\arg\max Q[\text{state}]$, the terminal is cleared, the ASCII grid is printed together with the action taken and reward received, and a 0.5-second delay is inserted so the movement can be followed.

4.6 Visualisation Module

To analyse learning, the training loop logs the total reward, number of steps and ϵ of every episode. Matplotlib is then used to draw (i) the reward and step learning curves with a 100-episode moving average, (ii) the ϵ -decay schedule, (iii) a per-block learning-progress bar chart, (iv) heatmaps of the maximum Q-value and greedy action for selected passenger/destination pairs, and (v) histograms of reward and steps over 200 greedy evaluation episodes. The trained agent's behaviour is additionally captured as RGB frames, which are assembled into a storyboard and an animated GIF.

5. Results and Discussion

5.1 Training Behaviour

With a linear decay of 0.001 per episode, ϵ falls from 1.0 to its floor of 0.01 after about 990 episodes (Figure 3). During this early phase the agent mostly acts randomly and obtains very poor returns, around -800 per episode at the start, because of long wandering and repeated illegal pickup/drop-off actions. As the Q-table fills in, the 100-episode moving average climbs steeply and turns positive within roughly the first 1,500 episodes, then stays stable for the rest of training (Figure 1).

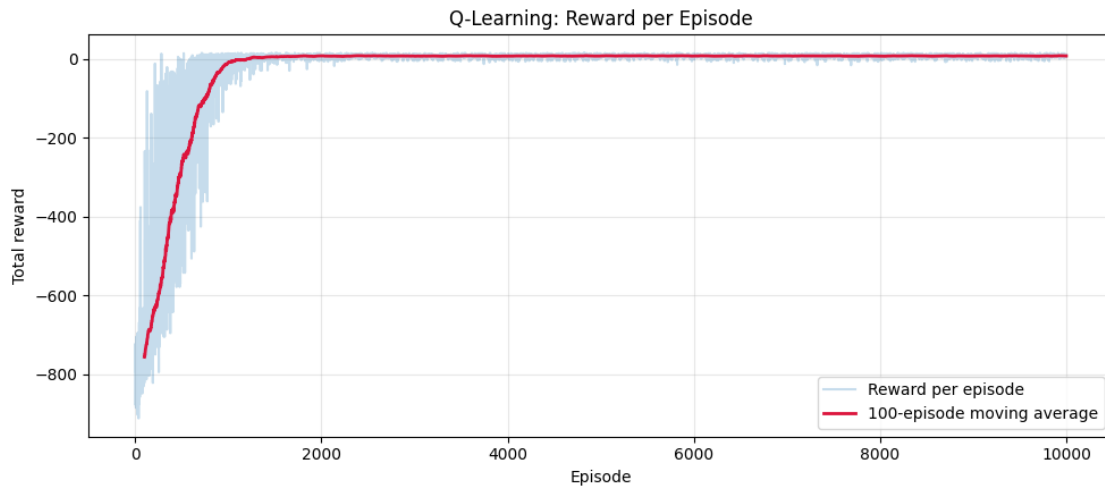


Figure 1: Total reward per episode with 100-episode moving average

The number of steps per episode shows the matching trend (Figure 2): episodes that initially run for hundreds of steps shrink to a short, stable route length. Figure 4 summarises the learning progress as the percentage of positive-reward episodes in each block of 1,000 episodes.



Figure 2: Steps per episode (log scale) with moving average

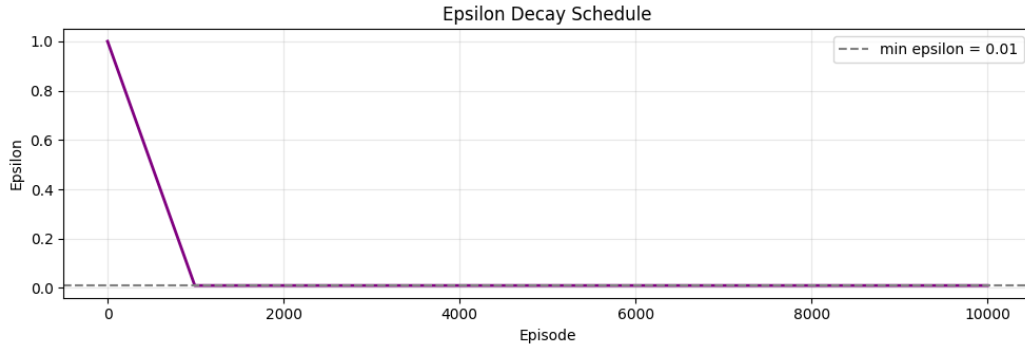


Figure 3: Epsilon decay schedule

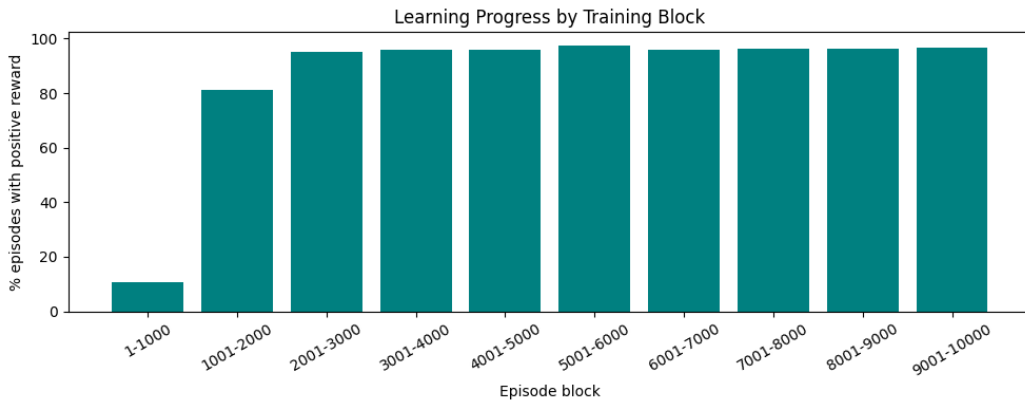


Figure 4: Percentage of positive-reward episodes per 1,000-episode block

5.2 Learned Values and Policy

Figure 5 visualises the learned Q-table for two situations. Each cell shows the highest Q-value for a taxi at that grid position and an arrow for the greedy action. In the left panel (passenger waiting at R, destination G) the arrows lead the taxi around the internal walls towards R, where the pickup action is chosen. In the right panel (passenger already in the taxi, destination B) values increase steadily towards B, where the greedy action is Dropoff and the value approaches +20, the terminal reward. The smooth gradient of values shows that the discount factor $\gamma = 0.99$ successfully propagates the final reward back through the route.

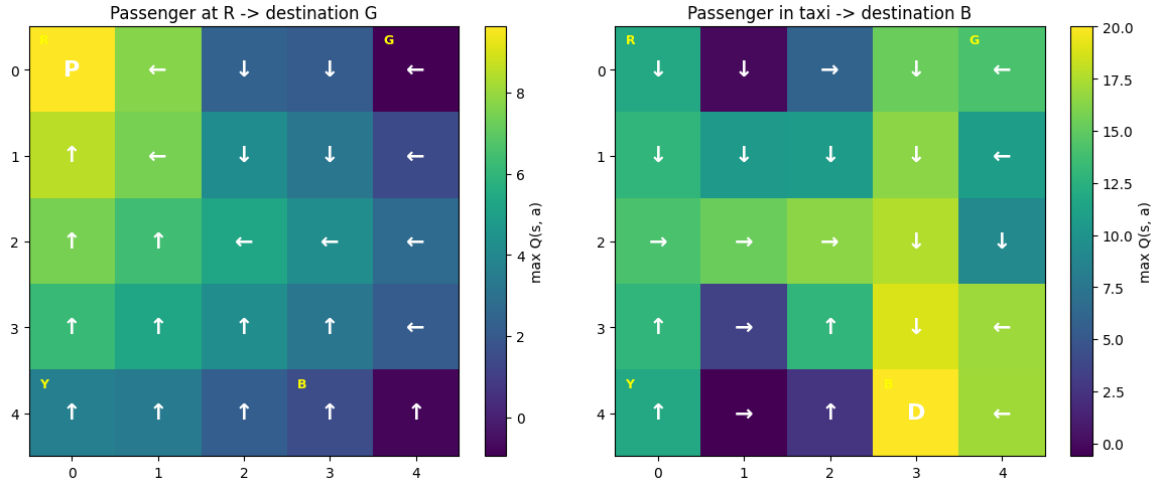


Figure 5: Heatmap of max Q -value and greedy action for two passenger/destination configurations

5.3 Evaluation over 200 Episodes

To go beyond a single run, the greedy policy (no exploration) was evaluated for 200 episodes starting from random configurations.

Metric	Value
Episodes evaluated	200
Success rate (passenger delivered)	100%
Mean total reward	7.97
Mean steps per episode	13.03
Range of steps	about 6 to 19

Table 5: Evaluation summary of the greedy policy

Every evaluation episode ended with a successful drop-off. The spread in reward and steps (Figure 6) is explained by the random start: some passenger and destination pairs are simply farther apart than others, not by agent errors.

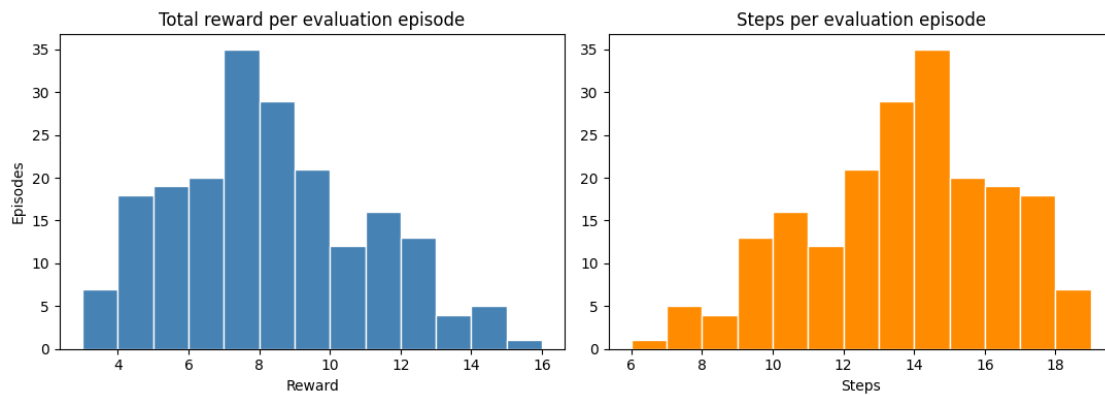


Figure 6: Distribution of total reward and steps over 200 evaluation episodes

5.4 Behaviour of the Trained Agent (Animation Storyboard)

A printed report cannot play an animation, so the run of the trained agent is shown as a frame-by-frame storyboard (Figures 7 and 8, with the step-by-step trace in Table 6). The episode starts with the taxi in the second row, the passenger at G (green, top-right) and the destination B (blue, bottom). The complete animation is supplied with the report as `taxi_agent_animation.gif` and can be replayed from the last cell of the notebook.

Step	Action	Reward	Observation
1	South (0)	-1	Moves down a row to get below the internal wall
2–3	East, North	-1 each	Crosses to the right-hand side of the wall
4–6	East, North, East	-1 each	Reaches the passenger location G
7	Pickup (4)	-1	Passenger boards the taxi
8–9	South $\times 2$	-1 each	Heads down the right-hand column
10	West (3)	-1	Aligns with the column of destination B
11–12	South $\times 2$	-1 each	Travels down to B
13	Dropoff (5)	+20	Passenger delivered; episode ends

Table 6: Step-by-step trace of the storyboard episode

The agent completed the task in 13 steps with a total return of $12 \times (-1) + 20 = +8$, with no illegal pickup or drop-off. For this starting configuration 13 steps is the minimum (6 moves to G, 1 pickup, 5 moves to B, 1 drop-off), so the learned policy is optimal here.

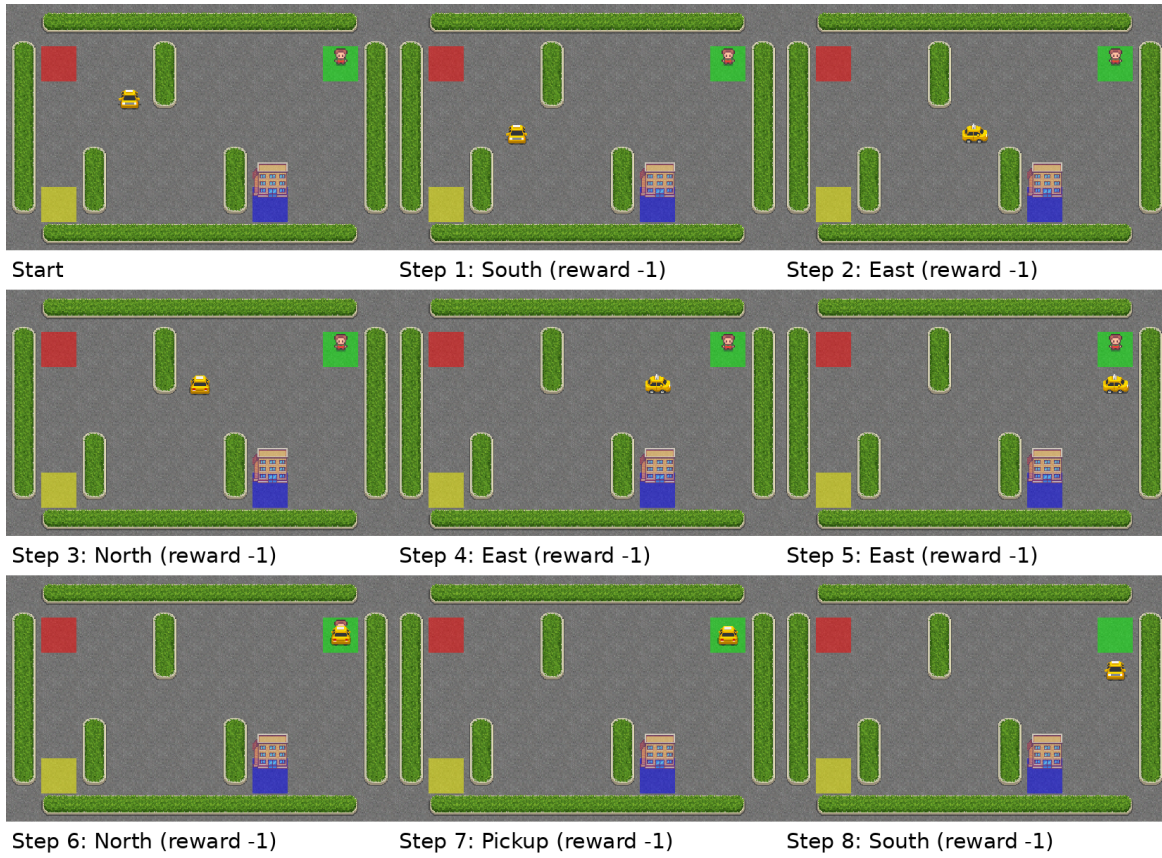


Figure 7: Storyboard, start to step 8 (taxi moves to the passenger and picks them up)

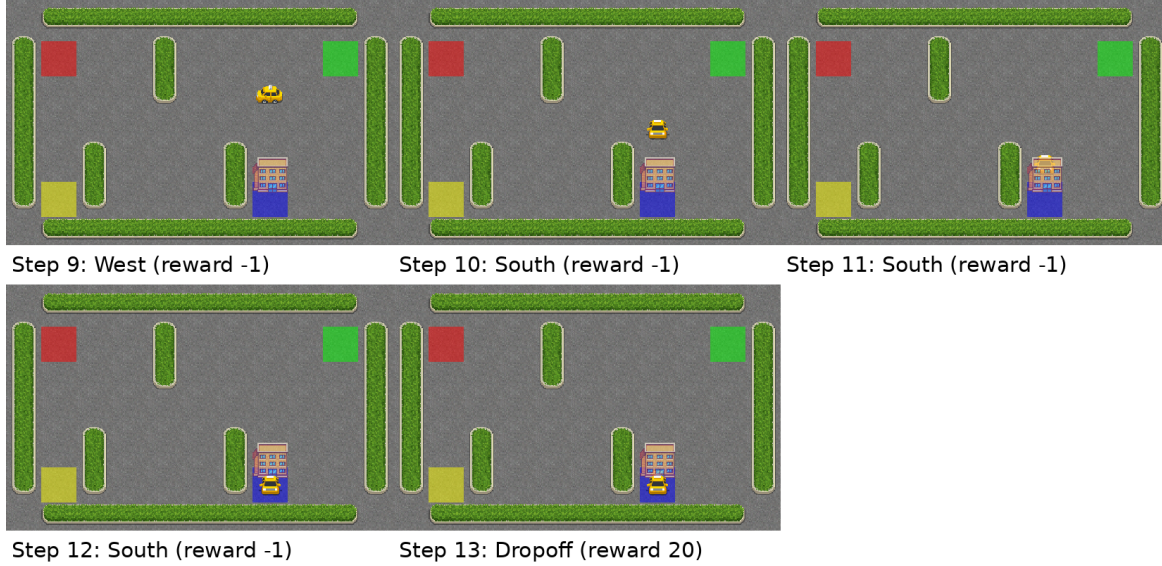


Figure 8: Storyboard, steps 9 to 13 (taxi carries the passenger to B and drops them off)

5.5 Discussion

- The Q-table successfully encodes the full task: route planning around walls, the pickup–drop-off ordering and avoidance of the -10 penalty.
- The learning curves show rapid convergence: ϵ -greedy exploration for the first $\sim 1,000$ episodes is enough to find a stable policy in this 500-state problem.

- A 100% success rate over 200 random starts, with a mean of about 13 steps, indicates that the policy generalises over the whole state space and not just to one configuration.
- Tabular methods are highly effective here because the state space is small and fully enumerable.

6. Observations and Limitations

The following points were noted during review of the code and documentation and are recommended for correction or further study:

- **Environment version mismatch.** The README states Taxi-v3, while the notebook uses Taxi-v4. The current Gymnasium release deprecates v3, so the README should be updated to v4.
- **Script name.** The README instructs running `taxi_agent.py`, whereas the submitted file is a notebook; a script should be exported or the README updated.
- **Epsilon schedule description.** The README says exploration shifts gradually over 10,000 episodes; in fact, the linear schedule reaches the 0.01 floor at about episode 990. A slower or exponential decay could be tested.
- **Terminal-state bootstrapping.** The update always adds $\gamma \cdot \max Q(s')$, even when the episode has ended. Setting the future term to zero on terminal transitions is the standard formulation.
- **Single training run.** Results come from one random seed; repeating training over several seeds would give confidence bands on the learning curves.

Three points raised during the initial review have been addressed in the final notebook: the loops now stop on terminated or truncated, evaluation is carried out over 200 episodes instead of one, and reward and step learning curves are logged and plotted.

7. Conclusion and Future Scope

7.1 Conclusion

This project demonstrated that a simple tabular Q-Learning agent, built from scratch with NumPy, can learn an optimal policy for the Taxi grid-world within seconds of training. The work illustrates the Bellman update, the exploration–exploitation trade-off through ϵ -greedy decay, and the effect of reward shaping on learned behaviour. Evaluation over 200 episodes gave a 100% success rate with a mean of about 13 steps, and the visualisations and animation storyboard confirm that the learned behaviour is sensible and near-optimal.

7.2 Future Scope

- Save and load the trained Q-table (as a .npy file) so retraining is not required on each run.
- Add a hyperparameter search over α , γ and ϵ -decay, and compare linear versus exponential decay.
- Repeat training over multiple random seeds and plot confidence bands on the learning curves.
- Compare Q-Learning with SARSA and with dynamic-programming baselines such as value iteration.
- Extend to a Deep Q-Network (DQN) in PyTorch for continuous or large state spaces.

References

- [1] Sutton, R. S. and Barto, A. G., Reinforcement Learning: An Introduction, 2nd ed., MIT Press, 2018.
- [2] Watkins, C. J. C. H. and Dayan, P., “Q-learning,” Machine Learning, vol. 8, pp. 279–292, 1992.
- [3] Dietterich, T. G., “Hierarchical Reinforcement Learning with the MAXQ Value Function Decomposition,” Journal of Artificial Intelligence Research, vol. 13, pp. 227–303, 2000.
- [4] Farama Foundation, Gymnasium Documentation – Taxi environment.
<https://gymnasium.farama.org/>
- [5] NumPy Documentation. <https://numpy.org/doc/>

Appendix A: Project Files

File	Description
taxiagent.ipynb	Original notebook: training loop and evaluation/animation
taxiagent_with_visualizations.ipynb	Extended notebook with learning curves, heatmaps and 200-episode evaluation
taxi_agent_animation.gif	Animated GIF of the trained agent's episode (storyboard source)
readme.md	Project overview, setup instructions and future improvements